



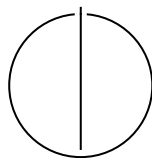
SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Formalization of the TCG MARS
Specification in Rocq**

Junguo Xu





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

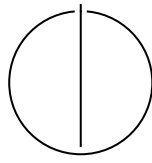
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Formalization of the TCG MARS
Specification in Rocq**

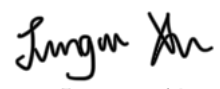
**Formalisierung der
TCG-MARS-Spezifikation in Rocq**

Author:	Junguo Xu
Examiner:	Prof. Dr. Pramod Bhatotia
Supervisor:	Julian Pritzi
Submission Date:	25.09.2026



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 25.09.2026


Junguo Xu

Acknowledgments

First, I want to thank my advisor, Julian Pritzi, for patiently providing support and constructive advice. His feedback was always helpful.

Then, I would also like to thank Prof. Dr. Pramod Bhatotia for giving me the opportunity to work on this thesis.

Lastly, I would like to thank my family and friends for supporting me during my studies.

Abstract

The Measurement and Attestation RootS (MARS) specification defines a lightweight Root of Trust (RoT) architecture for measurement, storage, and reporting. It supports remote attestation and helps ensure the integrity of small systems, such as microcontrollers or embedded systems. However, the MARS specification describes the logic of MARS devices in natural language, which makes it easier to read but might also make it ambiguous. Readers could interpret this specification differently, and it would be difficult to determine precisely how a MARS device should behave.

This thesis models MARS in the Rocq prover as a state machine. All the commands are modeled using a deterministic transition function. The transition function maps a device state and a command to a new state and a response code. We identify all the constants and functions the MARS specification assumes, and introduce them as a *Profile*. The properties and invariants then hold for every concrete *Profile*. We use the model to verify invariants, prove properties, and identify ambiguities or inconsistencies. The natural-language specification can then be translated into precise definitions.

The formalization covers all thirteen commands of the specification and proves nine groups of properties; for example, the Primary Seed (PS) is never modified, and secrets are never leaked to the host. We reported three important findings and five caveats, for example, the specification does not specify whether a non-sequence command should end the running sequence in failure mode. These results identify cases that require clarification and help the specification authors refine the MARS specification.

Contents

Acknowledgments	iv
Abstract	v
1 Introduction	1
1.1 Motivation	1
1.2 Research Gap	2
1.3 Problem Statement	2
1.4 The Approach	2
1.5 Contributions	3
2 Background	4
2.1 Remote Attestation	4
2.1.1 Passport Model	4
2.1.2 Background Check Model	5
2.2 The MARS Specification	5
2.2.1 Scope	5
2.2.2 MARS as a Root of Trust	6
2.2.3 Selectable Registers	7
2.2.4 Failure Mode	7
2.2.5 Key Hierarchy	7
2.2.6 Relationship between MARS and the Host	8
2.3 Interactive Theorem Prover	9
2.3.1 Rocq Prover	9
3 Overview	11
3.1 System Overview	11
3.2 Design Goals	11
3.3 System Workflow	13
3.4 System Components	13
4 Design	14
4.1 Modeling Approach: Transition Function	14

4.2	The State Model	14
4.3	Profile	15
4.4	CryptSnapshot [MARS, §5.6.9]	15
4.5	Command Interface	16
4.5.1	MARS_SelfTest [MARS, §8.1.1]	16
4.5.2	MARS_CapabilityGet [MARS, §8.1.2]	17
4.5.3	Sequence Primitives	17
4.5.4	MARS_PcrExtend [MARS, §8.3.1]	18
4.5.5	MARS_RegRead [MARS, §8.3.2]	18
4.5.6	MARS_Derive [MARS, §8.4.1]	18
4.5.7	MARS_DpDerive [MARS, §8.4.2]	19
4.5.8	MARS_PublicRead [MARS, §8.4.3]	19
4.5.9	MARS_Quote	19
4.5.10	MARS_Sign	20
4.5.11	MARS_SignatureVerify	20
4.6	MARS dispatcher and Check Ordering	21
5	Implementation	23
5.1	Profile	23
5.1.1	Profile Constants	23
5.1.2	Abstract Types	24
5.1.3	Support Functions	26
5.2	State Layer	27
5.2.1	MARS_STATE	27
5.2.2	Initialization	28
5.3	Command Interface	28
5.3.1	Snapshot	28
5.3.2	Response Code	29
5.3.3	Return Type	30
5.3.4	The Transition Function	30
5.4	Proof Engineering	31
6	Evaluation	33
6.1	Coverage of the Specification	33
6.2	Machine-Checked Properties and Invariants	34
6.2.1	PS Persistence	34
6.2.2	Register Count Invariance	35
6.2.3	Failure Mode	35
6.2.4	Sequence Discipline	35

6.2.5	Remaining Properties	36
6.3	Findings in the Specification	36
6.4	Limitations	38
7	Related Work	39
7.1	Formal Analysis of Trusted Hardware	39
7.2	Formalization of Specifications	39
7.3	Verified Implementation	39
8	Conclusion	41
8.0.1	Future Work	41
	Abbreviations	42
	List of Figures	44
	List of Tables	45
	Bibliography	46

1 Introduction

Computing technologies have become an indispensable part of modern society, supporting everything from personal devices to critical infrastructure. However, the widespread occurrence of cyberattacks makes it increasingly challenging to trust these systems. Once device properties are manipulated, sensitive information might be leaked [Mic25].

To address this evolving threat landscape, the Trusted Computing Group (TCG) has standardized several concepts related to RoT. RoT is a fundamental concept in hardware and software security; it provides a set of functions to enhance the security of the device in which it is integrated [Tru].

The best-known implementation of a RoT is the Trusted Platform Module (TPM); it is a hardware-based security component, and after more than 20 years' development, TPM has become the international standard [Tru26]. For larger devices such as computers and laptops, the TPM is usually the first choice. However, a full TPM design might be too complex, and its cost is too high if we just want to implement a RoT in smaller devices such as microcontrollers or embedded systems. For such smaller devices, the TCG defined the MARS specification that targets a more lightweight design. MARS is designed to be simple enough that we can implement it as a state machine [MARS].

1.1 Motivation

Despite MARS' lightweight character, precision is still important. The security level of MARS depends on the exact behaviors of every command defined by the specification. Because MARS is the Trusted Computing Base (TCB) core on which the security of the remainder of the system is built, any inconsistency could lead to vulnerabilities in the whole system. For that reason, the specification has to describe the logic precisely and ensure consistency. However, most specifications are written in natural language; this makes them easier for humans to read, but also creates problems: natural language can be ambiguous, may hide assumptions, and may contain inconsistencies. For security-critical systems, this is risky, because even a small ambiguity in the specification can lead to different implementations with conflicting behaviors.

1.2 Research Gap

Currently, evaluating such specifications often depends on manual review, expert analysis, and the implementation/testing of prototypes. These methods are useful, but they are limited. Traditional testing can only prove the presence of flaws, but it usually can not show that a class of bugs is impossible. For MARS specifically, there is no precise, formal, machine-checkable model of the core command behavior. Having precise formal semantics allows software tools to automatically analyze and simulate the specification. Therefore, we need a formal model so that we can use a theorem prover (we will use Rocq in this thesis) to prove some properties and refine the MARS specification.

1.3 Problem Statement

We ask the following research question:

How can we formally model the MARS state machine in Rocq to identify inconsistencies and prove invariants of the MARS specification?

We can split this question into three concrete sub-questions:

RQ1 Can the MARS state and its complete command interface be modeled faithfully?

RQ2 Which invariants and properties stated (or implied) by the specification can be proven?

RQ3 Which ambiguities or inconsistencies does the specification have?

1.4 The Approach

To formally model the MARS state machine, we need to analyze the MARS specification. Firstly, we identify all the constants and functions that the specification assumes; we introduce them as abstract natural numbers and functions. Then we define abstract types as input/output types for these functions. To build a state machine, we need to identify all state fields in MARS and define a transition function. For every command, this transition function maps the current state to the next state; we use pattern matching to combine all the commands into one transition function. Finally, we identify properties and invariants of the MARS specification and prove them. If we cannot prove a theorem, either our model is wrong, or we have misunderstood the specification. This requires refining the model, and we repeat this process until all the theorems are proven.

1.5 Contributions

This thesis provides the first machine-checkable model of the MARS specification. In summary, it makes the following contributions:

- We design and implement the first formal model of the MARS state machine in the Rocq prover.
- We state and prove nine groups of invariants and properties of the specification.
- We identify and document three findings and five caveats for the specification.

2 Background

Our system aims to formalize the MARS specification and prove its properties in the Rocq prover. A MARS is a RoT in remote attestation. To successfully develop a MARS state machine, we need to introduce remote attestation, the MARS specification, and interactive theorem provers in detail.

2.1 Remote Attestation

Remote attestation is a technique to verify the integrity of a system. This technique is widely used in different areas [Kim24]. Consider an entity that has confidential data and can release it to authorized devices. For any device that wants to retrieve confidential data, we need to know that this device has not been compromised. Remote attestation helps us to make sure that the device is trustworthy [Bir+23].

Remote ATtestation procedureS (RATS) is an architecture for remote attestation defined by the Internet Engineering Task Force (IETF); there are three main roles in this architecture [Bir+23]:

- The *attester* produces believable information about itself (*evidence*).
- The *verifier* appraises *evidence* via appraisal policies and creates the *attestation result*.
- The *relying party* decides whether or not to consider that Attester a trustworthy peer, using the *attestation result*.

To understand the structure of the attestation workflow, let us look at the following models.

2.1.1 Passport Model

The passport model describes attestation through an example of international travel (see Figure 2.1). A citizen (*attester*) submits documents (*evidence*), such as a birth certificate and biometric information to the host country (*verifier*). The host country checks whether the documents satisfy its policies. If the requirements are met, the host

country issues a passport (*attestation result*). The citizen can then present their passport to the foreign country's border patrol agent (*relying party*). The border patrol agent does not repeat the complete citizenship verification process. Instead, it checks whether the passport is authentic and whether the passport is issued by a trusted authority [Mic25].

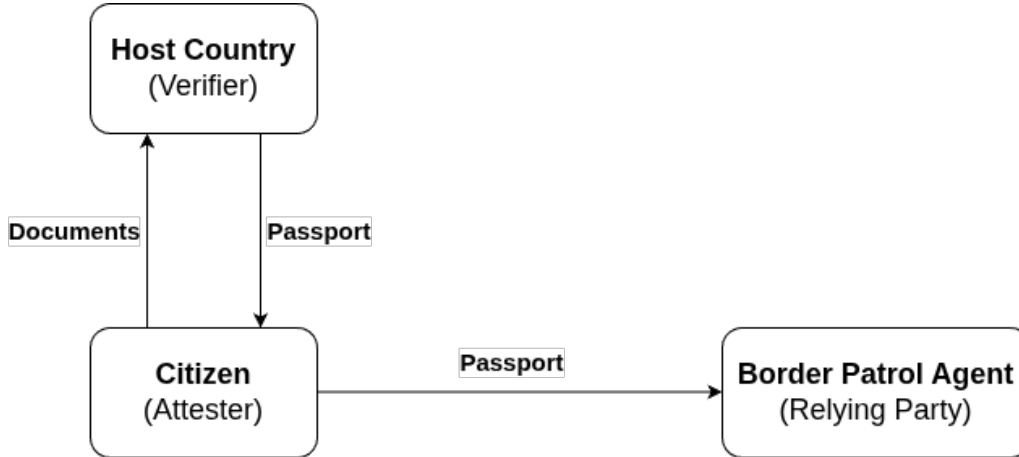


Figure 2.1: Passport Model [Mic25]

2.1.2 Background Check Model

The background check model gives another workflow (see Figure 2.2). A person (*attester*) applies for a job and provides information about their educational background (*evidence*) to the employer (*relying party*). The employer then forwards this information to the school (*verifier*) for evaluation. If the educational background is authentic, the school produces an *attestation result* and gives it to the employer. After receiving the *attestation result*, the employer checks if the result is authentic and if the result is produced by a trusted school [Mic25].

2.2 The MARS Specification

The MARS specification is defined by TCG, and it focuses on establishing a RoT for resource-constrained devices [Tru21].

2.2.1 Scope

The TCG MARS specification describes the logic required for devices so that they can perform attestation. The specification does not describe how MARS and the host

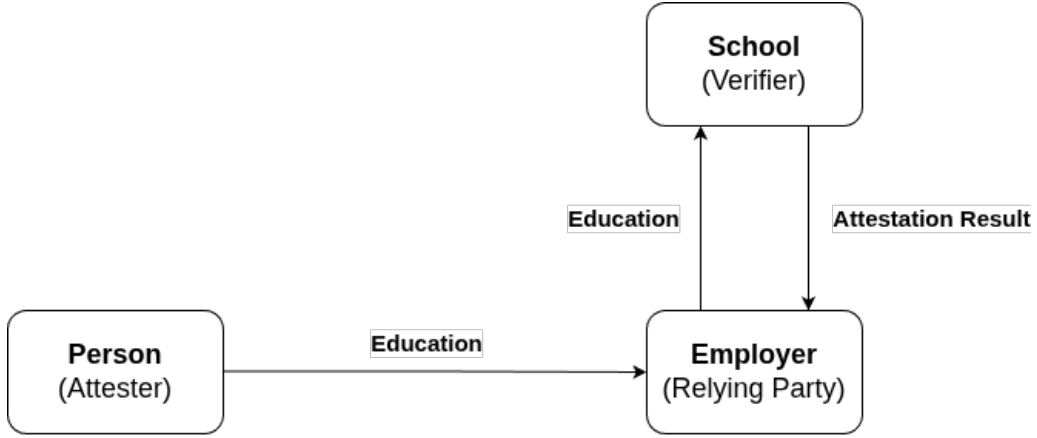


Figure 2.2: Background Check Model [Mic25]

communicate; it leaves implementation details, such as register count or cryptographic algorithms, to a so-called platform-specific *Profile*. Manufacturers can implement MARS on different platforms, and MARS should be parameterized with a *Profile* [MARS].

The MARS specification assumes some internal functions (they are also called support functions in the specification because they are referenced in commands) that are inaccessible outside of MARS, and the specification defines some commands which can be invoked by the host. These commands are defined with behavioral descriptions, and they invoke internal functions. In the specification, all the names of functions start with `Crypt`, and all the names of commands start with `MARS_`. The MARS specification also provides standard C code for these commands and an official emulator for MARS [MARS]. Our target is to formalize the specification, so we model the state machine faithfully according to the MARS specification. When there is an ambiguity, we state it explicitly as a finding. If the ambiguity needs to be resolved to proceed, we refer to the official emulator and our own best judgment to infer the most likely intended meaning.

2.2.2 MARS as a Root of Trust

MARS defines a RoT for measurement, storage, and reporting:

Root of Trust for Measurement (RTM) The Root of Trust for Measurement (RTM) loads and measures a code or data module (which is called an event in the MARS specification [MARS, §4.1]); the measurement is a cryptographic digest and is then delivered to the Root of Trust for Storage (RTS). Optionally, the measurement can be

inserted in an event log. After that, the module will be executed or processed and will be responsible for the same actions as the RTM [MARS, §4.2].

Root of Trust for Storage (RTS) Instead of storing all the event digests, the RTS securely maintains cumulative digests in its Platform Configuration Registers (PCRs) [MARS, §4.3].

Root of Trust for Reporting (RTR) The host of MARS can invoke some commands with a bitmask parameter named `regSelect` [MARS, §5.3.4.1] to ask the Root of Trust for Reporting (RTR) to sign one or more register values with an Attestation Key (AK). The RTR then provides evidence of the measured state to an external verifier; the evidence will be evaluated [MARS, §4.4].

2.2.3 Selectable Registers

There are two types of selectable registers; they can be selected via `regSelect` in some commands. PCRs cannot be written directly; they can only be extended: the new value is the hash of the old value concatenated with the measurement [MARS, §5.3.4.2]. In contrast to PCRs, Trusted Sensor Registers (TSRs) can not be extended; they are linked to onboard sensors (or clocks, etc.). When a TSR is selected in a command, the freshly sampled value of the associated sensor will be written to this TSR. TSRs can only be modified when they are selected [MARS, §5.3.4.3].

2.2.4 Failure Mode

A *Profile* may require that certain parts of MARS be tested before they are used. We could do either all available tests during system initialization or only the tests available for logic required by the issuing command. MARS enters failure mode when any test fails or any other internal error is detected [MARS, §5.3.1].

2.2.5 Key Hierarchy

MARS uses secrets to generate other secrets, keys, etc. The key hierarchy is rooted in the PS. The PS is a persistent secret and is the most critical security parameter [MARS, §5.3.2]. The initial Derivation Parent (DP) can be derived from the PS, using an internal function `CryptDpInit` [MARS, §5.6.8]. DP is a volatile secret from which other keys/secrets are derived [MARS, §5.3.3]. Except for the initial DP, all other keys/secrets are derived by a Key Derivation Function (KDF) from a DP, a label, and a context [MARS, §5.6.5]. With different labels, keys with different usages are generated. Table 2.1

shows all the labels and the corresponding keys, and Figure 2.3 shows the key hierarchy with all four labels [MARS, §5.5].

Table 2.1: Cryptographic Key Labels

Label	Description
MARS_LX	bytes for external use.
MARS_LD	derivation Parent
MARS_LU	unrestricted signing key that signs an externally supplied digest
MARS_LR	restricted attestation key that signs a snapshot of selected registers

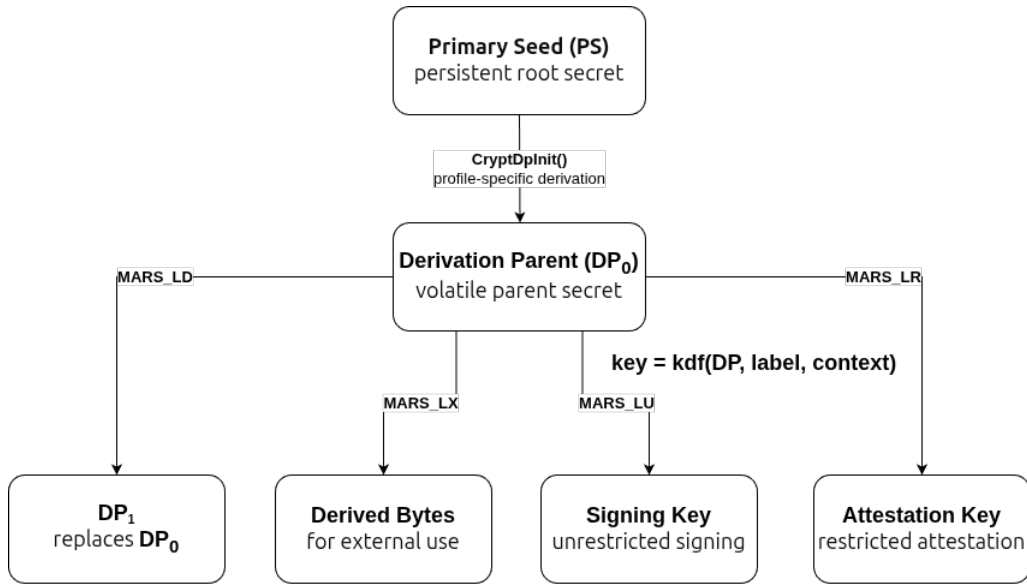


Figure 2.3: MARS Key Hierarchy

2.2.6 Relationship between MARS and the Host

The MARS specification allows attaching a MARS device to its host in different ways. For example, MARS can reside within a host microcontroller; it can also be logically attached via interprocess communication. Manufacturers can make their own choice [Tru23].

The MARS and the host are separate but cooperating entities. The RTM is not a part of MARS; it is located on the host, measures events, and sends the measurements to the MARS. The MARS device needs to implement security primitives for hashing, signing,

and key derivation [MARS, §5.1]. In the MARS specification, symmetric cryptography is necessary, but we can also optionally support asymmetric cryptography [Tru25]. In our thesis, we focus on the asymmetric *Profile* because we want to model as many features and commands as possible so that our model can be more powerful.

MARS is attached to the host as a RoT that provides storage and reporting functionality. The MARS specification defines a host as a computing platform with an attached MARS device [MARS]. In MARS, internal functions are critical to the security and therefore inaccessible outside of MARS. The host needs to use the command interface defined in the specification to communicate with the MARS device: The host can invoke `MARS_PcrExtend` [MARS, §8.3.1] to extend the PCR values in MARS; it can invoke `MARS_Quote` [MARS, §8.5.1] to ask the MARS for a snapshot of the selected registers.

Figure 2.4 shows how the verifier, the host and the MARS interact with each other: the host invokes `MARS_PcrExtend` [MARS, §8.3.1] to extend the PCR values in MARS; the verifier challenges the device with a nonce; the host invokes `MARS_Quote` [MARS, §8.5.1] to ask MARS for a signed snapshot of the selected registers; the evidence will then be presented to the verifier; once the evidence is authentic, the verifier then produces an attestation result.

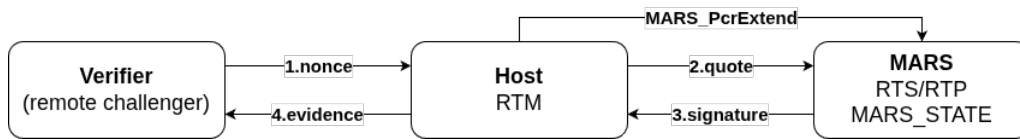


Figure 2.4: Verifier-Host-MARS interaction model

2.3 Interactive Theorem Prover

An Interactive Theorem Prover (ITP) is a software tool that helps humans develop formal proofs: a human writes definitions and proofs; the machine checks every step. Proofs can be constructed interactively through a dialog between the user and the assistant.

2.3.1 Rocq Prover

There are many different proof assistants such as Isabelle or Lean; Lean is widely used in the formalization of mathematical theorems because of its large cohesive library. But in our thesis, Rocq Prover is selected because Rocq has a mature ecosystem for verifying security-critical and low-level systems, while Lean is more developed for mathematics.

Rocq Prover is one of the most mature ITPs [The]; it is a functional programming language and has been used for projects such as the CompCert compiler [Ler09] and the formalization of the four-color theorem [Gon08].

In Rocq, every function is total; this means every function must terminate and must be defined on all inputs. This is exactly the property that makes hidden gaps in a specification visible: a partial English description must be turned into a total function, so we need to make an explicit decision for every unspecified case. A typical Rocq development contains `Record`, `Inductive`, `Definition`, and `Theorem`.

Record puts multiple components into a single type. Each component is called a field, and it can be accessed with a corresponding projection function.

Inductive defines data by listing its possible constructors. Rocq's pattern matching must cover all cases.

Definition defines a function. It can be used in other operations in Rocq to make our system more structured and easier to understand.

Theorem states a goal we want to prove, and it will be proven with tactics. A full proof ends with `Qed` and can also be closed with `Admitted`. A clean development must not contain any `Admitted`.

3 Overview

Our system turns the natural-language MARS specification into a Rocq state machine. The artifact is not a replacement for the standard C code in the specification, and it is not an emulator for any devices. Its purpose is to make the MARS specification machine-checkable.

3.1 System Overview

Figure 3.1 shows the two layers of the system and how they relate to the specification and to the Rocq kernel.

The specification is encoded section by section into the model layer and property layer. The model layer contains the *Profile*, the MARS state, and the transition function. The property layer contains invariants and properties stated or implied by the specification. They are presented as theorems and are proven; the Rocq kernel checks all proofs. Decisions we made become findings (see Section 6.3) in the specification.

3.2 Design Goals

The design of this model follows three goals:

- G1 Faithfulness** Each definition corresponds to a section in the specification. According to the comments, readers can find the section number of each definition easily and audit the model against the specification line by line.
- G2 Generality** The system should be parameterized with *Profile*; all the theorems hold for every MARS *Profile* that satisfies the constraints specified in our model.
- G3 Determinism and totality** The transition function is deterministic and total. Every command in every state has exactly one deterministic result. This property exposes inconsistencies of the specification.

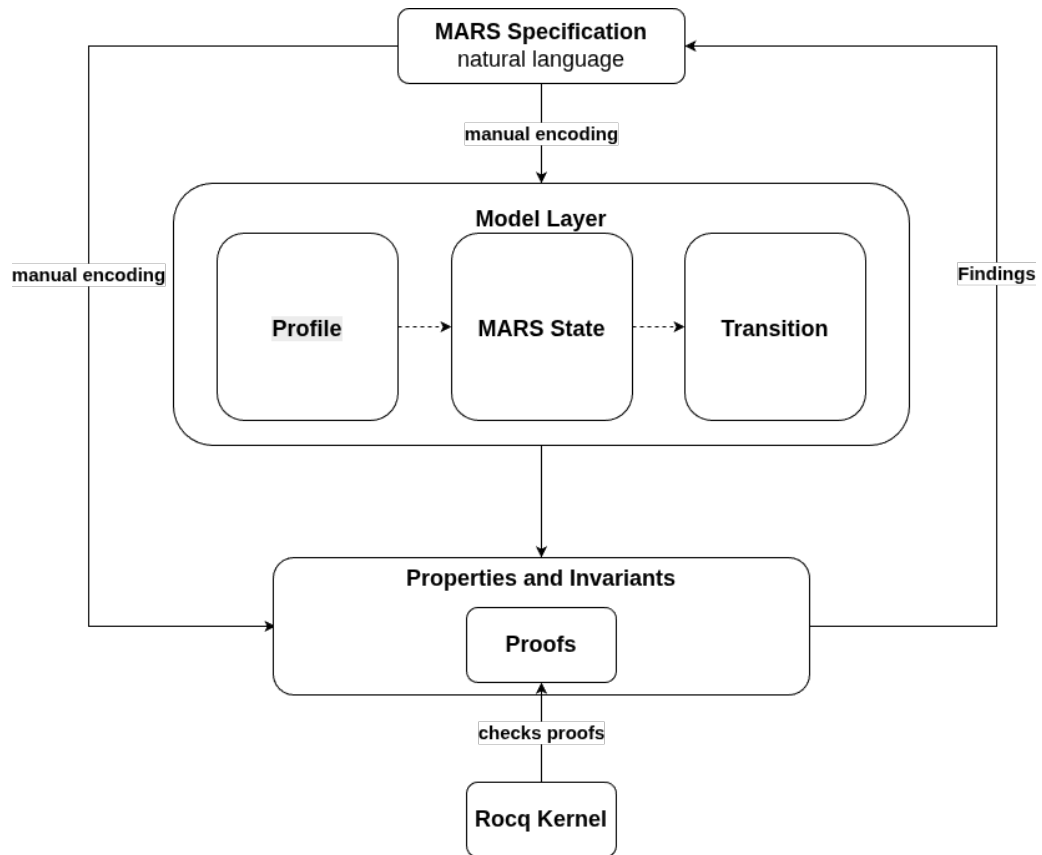


Figure 3.1: System Overview

3.3 System Workflow

The development follows an iterative workflow:

1. **Read and encode.** We read a section of the specification and encode its data (constants) and behavior (functions).
2. **Make decisions.** Where the text is silent, we have to make a decision. Each decision is an interpretation we chose and is recorded as a potential finding.
3. **State properties.** We turn every invariant stated in the specification into a formal theorem statement.
4. **Prove.** We prove the theorems. Once a proof attempt fails, then we know either our model is wrong, or the specification has inconsistencies, which leads to a finding.
5. **Iterate.** Findings sometimes require refining the model or the property statements; we repeat until all properties are proven.

3.4 System Components

The development consists of four components:

Profile Layer contains three parts: profile constants, abstract types, and support functions. They are put into Records for a clear structure. The profile constant Record contains all the profile constants [MARS, §6.1]. These constants are introduced as natural numbers. The abstract type Record contains all the abstract types we defined. These types are input/output types of functions. The support function Record introduces support functions [MARS, §5.1]. They are cryptographic functions, and we leave the concrete implementations abstract.

State Layer contains the MARS state Record with state fields; for each field, there is a dedicated update function [MARS, §5.3]. It also contains the MARS initialization: all the state fields are reset to the initial values, and self-test is performed [MARS, §5.4].

Command Interface contains a per-command transition function; for every command it maps the current MARS state to the next state.

Property Layer contains helper lemmas and nine groups of theorems (P1–P9). All the theorems are fully proven and end with Qed.

4 Design

This chapter explains and motivates the design of a MARS state machine: the transition function, the MARS state, and the command interface.

4.1 Modeling Approach: Transition Function

Our model is a state machine; its core is a transition function δ . Every command invoked by the host could change the state of MARS. Besides that, some commands select TSRs, which trigger sampling and change the register values of MARS. The transition function in our model has the following format:

$$\forall c \in C, \delta_c : S \times E \rightarrow S \times R \times (O_c \cup \{\perp\}), \delta_c(s, e) = (s', r, o).$$

The transition function δ_c of each command maps the current state $s \in S$ and sampled values of the current environment $e \in E$ (which will be written into TSRs when they are selected with `regSelect`) to the next state $s' \in S$, a response code $r \in R$, and an output $o \in (O_c \cup \{\perp\})$.

A response code shows if the invoked command ran successfully or what problem MARS had while executing the command; every MARS command must return a response code [MARS, §6.2]. All the names of response codes start with `MARS_RC_`.

Besides response codes, commands can also return an output. Different commands return different types, so we use a command-dependent output type: for each command c , we compute the correct output type O_c . Some commands do not produce output, and some can not return output when they fail. Therefore, each command may also return nothing, i.e., $\perp$.

This function is total. For every command and every state, there is a deterministic outcome. For every case that the specification does not cover, we have to make a decision and document our interpretation.

4.2 The State Model

The MARS specification gives some state fields: failure mode, PS, DP, PCR and TSR values [MARS, §5.3].

In many commands, parameters are concatenated together and hashed to produce a digest; the concatenation process is called a sequence. MARS provides sequence commands to start, update, and complete a sequence. Any running sequence should be ended immediately if a non-sequence command is invoked [MARS, §8.2]. In our model, we should then show if any sequence is running, and which data are already concatenated together.

Our state machine then contains the following fields:

- **failure** is the failure mode flag; it shows whether the MARS device is in failure mode.
- **PS** is the root secret; PS can not be modified, and it is not directly exposed through the command interface by any commands.
- **DP** shows the current DP; it can be replaced with a new value, but it is also not exposed.
- **regs** contains the value of all PCRs and TSRs. The order of registers should be PCR 0,..., TSR 0,... [MARS, §5.3.4].
- **shc** shows if there is any hash sequence in progress. We denote $\perp$ as no sequence started.

4.3 Profile

As discussed in Chapter 2, MARS can be implemented on different platforms; a designer should refer to the platform-specific *Profile* to develop a MARS device. The *Profile* contains profile constants that specify things like register count or digest length [MARS, §6.1]. A MARS provides some internal functions: hash-related functions, key derivation functions, etc. The MARS specification assumes them, and the implementation details also depend on the *Profile*. We should abstract these constants and functions, so that the theorems we proved can hold for different *Profiles*, which is our design goal G2 (see Section 3.2).

4.4 CryptSnapshot [MARS, §5.6.9]

Some MARS commands select registers by giving a bitmask `regSelect` [MARS, §5.3.4.1]; the parameter `regSelect` is used in the internal function `CryptSnapshot`. Unlike other internal functions, the MARS specification defines the behavior of this function, so in our model, we also define this function instead of abstracting it. `CryptSnapshot`

takes a `regSelect` and a context from commands, and then produces a snapshot: the sampled values of the current environment e are written into the selected TSRs; `regSelect`, values of the selected registers, and context are concatenated and then hashed.

$$\text{snapshot} = \text{CryptHash}(\text{regSelect} \parallel \text{REG\#} \parallel \dots \parallel \text{REG\#} \parallel \text{context}).$$

$$s' = s[\text{regs} \leftarrow e].$$

4.5 Command Interface

A MARS device provides a command interface. Some commands have to be implemented to support basic measurement and attestation, while some commands are not necessary and add complexity. In the MARS specification, the commands are divided into three categories: mandatory, recommended, or optional [MARS, §7]. Our model should support all the commands so that it can cover more cases. Table 4.1 shows all the commands defined by the MARS specification.

To get a better understanding of the command interface, we will introduce the commands in detail and show how they change the MARS state in different cases. In failure mode, all the commands (except for `MARS_CapabilityGet`) are rejected; the commands do not change the MARS state, and return `MARS_RC_FAILURE` and no output. For any invalid register index or `regSelect`, the commands do not change the MARS state, and return `MARS_RC_REG` and no output. We will skip these redundant cases for brevity.

Non-sequence commands end any running sequences, and then run as usual; `shc` becomes $\perp$ no matter if there is any sequence in progress. For brevity, we denote $\bar{s}$ as $s[\text{shc} \leftarrow \perp]$. In the following, if a non-sequence command only ends the running sequence and does not change other state fields, we say this command does not change the state.

4.5.1 MARS_SelfTest [MARS, §8.1.1]

`MARS_SelfTest` invokes `CryptSelfTest` [MARS, §5.6.1] to perform a self-test. When any test fails, the command returns `MARS_RC_FAILURE` and the MARS enters failure mode; otherwise, the command returns `MARS_RC_SUCCESS`.

$$\delta_{\text{SelfTest}}(s, e) = \begin{cases} (\bar{s}[\text{failure} \leftarrow \text{false}], \text{RC}_{\text{success}}, \perp), & \text{if } \text{CryptSelfTest} = \text{true}, \\ (\bar{s}[\text{failure} \leftarrow \text{true}], \text{RC}_{\text{failure}}, \perp), & \text{if } \text{CryptSelfTest} = \text{false}. \end{cases}$$

Table 4.1: MARS Commands (the prefix is removed for brevity)

Command	Description
SelfTest	Performs self-tests
CapabilityGet	Returns the <i>Profile</i> constant wanted by the host.
SequenceHash	Starts a hash sequence.
SequenceUpdate	Adds a data chunk into the sequence.
SequenceComplete	Ends the sequence and returns the resulting digest.
PcrExtend	Extends the selected PCR with a supplied digest.
RegRead	Returns the value of the selected register.
Derive	Returns bytes for external use.
DpDerive	Replaces the current DP with a new value of DP
PublicRead	Returns the public portion of the specified key.
Quote	Signs a snapshot of selected registers using a restricted attestation key.
Sign	Signs an externally supplied digest using an unrestricted signing key.
SignatureVerify	Verifies a digital signature using a derived verification key.

4.5.2 MARS_CapabilityGet [MARS, §8.1.2]

In failure mode, MARS_CapabilityGet is the only command that could run successfully; all the other commands are rejected and return MARS_RC_FAILURE. This command returns a profile constant according to the given Property Tag (PT) if the PT is found, and returns MARS_RC_VALUE otherwise. In this process, no state is changed.

$$\delta_{\text{CapabilityGet}}(s, e) = \begin{cases} (\bar{s}, \text{RC}_{\text{success}}, \text{Capability}(\text{PT})), & \text{if PT is found,} \\ (\bar{s}, \text{RC}_{\text{value}}, \perp), & \text{if PT is not found.} \end{cases}$$

4.5.3 Sequence Primitives

As we said, MARS provides some commands for sequences. MARS_SequenceHash invokes CryptHashInit to initialize the sequence hash context shc and starts a sequence; MARS_SequenceUpdate updates the sequence by adding a data chunk into shc; MARS_SequenceComplete ends the running sequence and computes a hash with CryptHashFinal.

MARS_SequenceUpdate and MARS_SequenceComplete can only work successfully when the sequence is already started, or they return MARS_RC_SEQ. MARS_SequenceUpdate is able to produce additional output, but no examples are shown in the specification, and it is also not implemented in the normative C code, so in our model, this command does not return any output. A set of sequence commands should not be interrupted by any other MARS commands; any other commands end the sequence immediately.

$$\delta_{\text{Hash}}(s, e) = (s[\text{shc} \leftarrow \text{CryptHashInit}], \text{RC}_{\text{success}}, \perp)$$

$$\delta_{\text{Update}}(s, e) = \begin{cases} (s, \text{RC}_{\text{seq}}, \perp), & \text{if shc} = \perp, \\ (s[\text{shc} \leftarrow (1 \parallel \text{data})], \text{RC}_{\text{success}}, \perp), & \text{if shc} = 1. \end{cases}$$

$$\delta_{\text{Complete}}(s, e) = \begin{cases} (s, \text{RC}_{\text{seq}}, \perp), & \text{if shc} = \perp, \\ (\bar{s}, \text{RC}_{\text{success}}, \text{CryptHashFinal}(1)), & \text{if shc} = 1. \end{cases}$$

4.5.4 MARS_PcrExtend [MARS, §8.3.1]

MARS_PcrExtend takes a pcrIndex and a digest as input; it does not produce any output. It changes the value of the selected PCR by extending it: the new value is the hash of the old value concatenated with the digest.

$$\text{result} = \text{CryptHash}(\text{regs}[\text{pcrIndex}] \parallel \text{digest}).$$

$$\delta_{\text{PcrExtend}}(s, e) = (\bar{s}[\text{regs}[\text{pcrIndex}] \leftarrow \text{result}], \text{RC}_{\text{success}}, \perp)$$

4.5.5 MARS_RegRead [MARS, §8.3.2]

MARS_RegRead takes a regIndex and returns the content of the selected register. It does not trigger sampling, and therefore does not change the state.

$$\delta_{\text{RegRead}}(s, e) = (\bar{s}, \text{RC}_{\text{success}}, \text{regs}[\text{regIndex}])$$

4.5.6 MARS_Derive [MARS, §8.4.1]

MARS_Derive first produces a snapshot from the input regSelect and context, then it invokes CryptSkdf [MARS, §5.6.5] to generate bytes for external use from the current DP, label MARS_LX, and the snapshot of the selected registers.

$$\text{snapshot} = \text{CryptSnapshot}(\text{regSelect}, \text{context}).$$

$$\text{output} = \text{CryptSkdf}(\text{DP}, \text{MARS_LX}, \text{snapshot}).$$

$$\delta_{\text{Derive}}(s, e) = (\bar{s}[\text{regs} \leftarrow e], \text{RC}_{\text{success}}, \text{output})$$

4.5.7 MARS_DpDerive [MARS, §8.4.2]

MARS_DpDerive is responsible for DP derivation; it can replace the current DP with a new value. This command has two modes. If the input context is not NULL, the command produces a snapshot from the input regSelect and context, then it replaces the current DP with a secret derived with CryptSkdf [MARS, §5.6.5] from the current DP, label MARS_LD, and the snapshot. With a NULL context, the DP is reset to the initial value using the PS and a *Profile*-specific algorithm CryptDpInit [MARS, §5.6.8]; in this mode, no snapshot or sampling is performed.

$$\text{snapshot} = \text{CryptSnapshot}(\text{regSelect}, \text{context}) \quad (\text{context} \neq \text{NULL}).$$

$$\text{result} = \text{CryptSkdf}(\text{DP}, \text{MARS_LD}, \text{snapshot}).$$

$$\delta_{\text{DpDerive}}(s, e) = \begin{cases} (\bar{s}[\text{DP} \leftarrow \text{CryptDpInit}(\text{PS})], \text{RC}_{\text{success}}, \perp), & \text{if context} = \text{NULL}, \\ (\bar{s}[\text{DP} \leftarrow \text{result}; \text{regs} \leftarrow e], \text{RC}_{\text{success}}, \perp), & \text{if context} \neq \text{NULL}. \end{cases}$$

4.5.8 MARS_PublicRead [MARS, §8.4.3]

MARS_PublicRead takes label MARS_LU or MARS_LR (other labels do not produce keys), and a context as input, and generates a key with CryptAkdf [MARS, §5.6.6] using the current DP, the label, and the context. This command does not change the state and returns the public portion of the specified key.

$$\text{key} = \text{CryptAkdf}(\text{DP}, \text{label}, \text{context}).$$

$$\delta_{\text{PublicRead}}(s, e) = (\bar{s}, \text{RC}_{\text{success}}, \text{public portion of the key})$$

4.5.9 MARS_Quote

MARS_Quote is used for attestation; it takes a regSelect, a nonce, and a context as input. It produces a fresh snapshot from the regSelect and the nonce. A restricted AK is generated with CryptXkdf [MARS, §5.6.7] using the current DP, label MARS_LR, and the context. The command then signs the snapshot with the AK and returns the signature.

$$\text{snapshot} = \text{CryptSnapshot}(\text{regSelect}, \text{nonce}).$$

$$AK = \text{CryptXkdf}(DP, \text{MARS_LR}, \text{context}) .$$

$$\text{signature} = \text{CryptSign}(AK, \text{snapshot}) .$$

$$\delta_{\text{Quote}}(s, e) = (\bar{s}[\text{regs} \leftarrow e], \text{RC}_{\text{success}}, \text{signature})$$

4.5.10 MARS_Sign

MARS_Sign is similar to MARS_Quote. It takes a context and a digest as input, and generates a signing key with CryptXkdf [MARS, §5.6.7] from the current DP, label MARS_LU, and the context. Then it signs the digest with the key and returns the signature. In this process, no snapshot or sampling is performed; the state is also not changed.

$$\text{key} = \text{CryptXkdf}(DP, \text{MARS_LU}, \text{context}) .$$

$$\text{signature} = \text{CryptSign}(\text{key}, \text{digest}) .$$

$$\delta_{\text{Sign}}(s, e) = (\bar{s}, \text{RC}_{\text{success}}, \text{signature})$$

4.5.11 MARS_SignatureVerify

To verify a signature, we have to provide the original materials (label MARS_LU or MARS_LR, and context) to generate the original key with CryptXkdf [MARS, §5.6.7]. MARS_SignatureVerify then invokes CryptVerify to verify the signature and returns the result.

$$\text{key} = \text{CryptXkdf}(DP, \text{label}, \text{context}) .$$

$$\text{result} = \text{CryptVerify}(\text{key}, \text{digest}, \text{signature}) .$$

$$\delta_{\text{SignatureVerify}}(s, e) = (\bar{s}, \text{RC}_{\text{success}}, \text{result})$$

4.6 MARS dispatcher and Check Ordering

In MARS, commands are assumed to be invoked by an internal MARS dispatcher; the dispatcher returns `MARS_RC_FAILURE` if the MARS is in the failure mode and the command is not `MARS_CapabilityGet`, returns `MARS_RC_COMMAND` if this command is not supported (we model all the commands so this response code is never returned), etc. (§8). The specification describes the behavior of each command, but not every check ordering is defined. In some cases, we have to make our own decision, and these decisions will be our findings.

Algorithm 1 shows in which order the dispatcher checks our commands.

Algorithm 1 Check Ordering

Require: state st , command cmd

- 1: **if** $st.failure$ **then**
 - 2: **if** cmd is `MARS_CapabilityGet` **then return** $(st, MARS_RC_SUCCESS, output)$
 - 3: **else return** $(st, RC_FAILURE, \perp)$
 - 4: **end if**
 - 5: **end if**
 - 6: **if** cmd is not a sequence command **then**
 - 7: $st \leftarrow st$ with $shc := \text{None}$
 - 8: **end if**
 - 9: execute the command action; return new state, response code, and output.
-

Figure 4.1 shows a state machine with two fields: the failure flag and the sequence context, and the transition behavior.

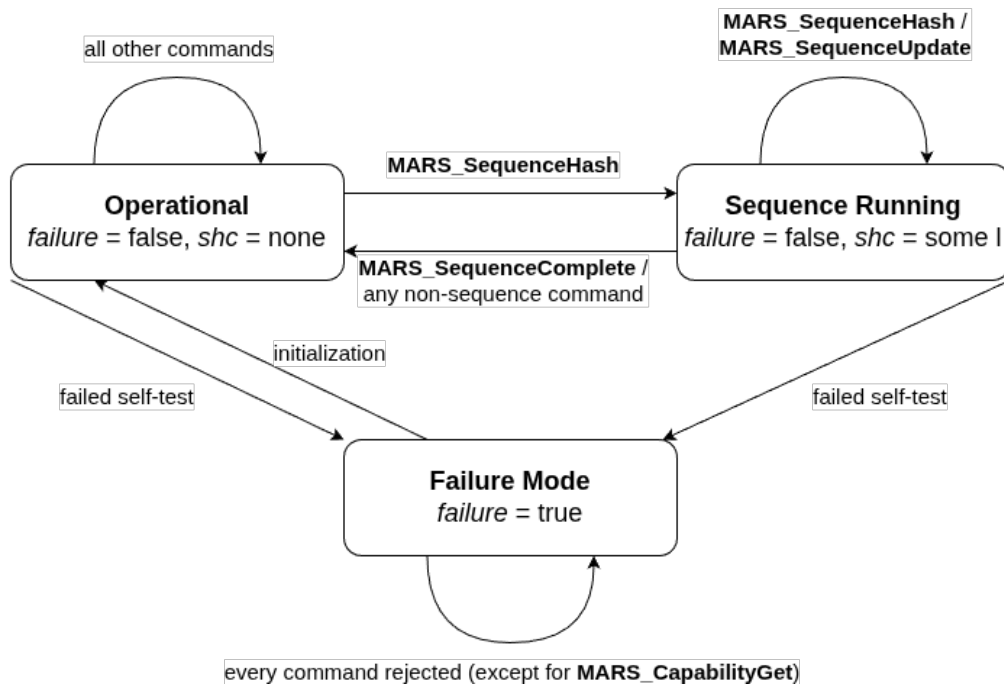


Figure 4.1: The Simplified MARS State Machine

5 Implementation

Our project is implemented in Rocq and uses only the standard library (Bool, Arith, NArith, Lists, and the Lia arithmetic tactic). It compiles with Coq and has no external dependencies. The code is organized in one file, `mars.v`, and it is divided into four parts: Profile Layer, State Layer, Command Interface, and Property Layer. Every definition is commented with the specification section it encodes.

5.1 Profile

The system should be parameterized with a *Profile*, so we need to identify parameters of a *Profile* such as constants or functions. To make the *Profile* more structured, we use Record to put parameters into different groups, and access each parameter with a projection function.

5.1.1 Profile Constants

PROFILE_CONSTANT (see Listing 5.1) contains all the *Profile* constants listed in the MARS specification [MARS, §6.1]: register count, lengths of digests, signatures, and keys, and algorithm identifiers. This type is restricted: a MARS must have at least one PCR; there can not be more than thirty-two registers.

```
1 Record PROFILE_CONSTANT : Type := {
2   PROFILE_COUNT_PCR : nat; (* "number of PCRs *"
3   PROFILE_COUNT_TSR : nat; (* "number of TSRs *"
4   PROFILE_COUNT_REG : nat := PROFILE_COUNT_PCR + PROFILE_COUNT_TSR;
5
6   pcr_ge_1 : 1 <= PROFILE_COUNT_PCR;
7   reg_le_32 : PROFILE_COUNT_REG <= 32;
8
9   PROFILE_LEN_DIGEST : nat; (* length of a digest *)
10  PROFILE_LEN_SIGN : nat; (* "length of a signature *"
11  PROFILE_LEN_KSYM : nat; (* "length of symmetric key produced by CryptSkdf
    *)
12  PROFILE_LEN_KPUB : nat; (* "length of public portion of an asymmetric key
    produced by CryptAkdF *)
```

```

13 PROFILE_LEN_KPRV : nat; (* "length of asymmetric key produced by
    CryptAkdF *)
14 PROFILE_LEN_XKDF : nat := PROFILE_LEN_KPRV; (* "length of key produced by
    CryptXkdf *)
15
16 PROFILE_ALG_HASH : nat;
17 PROFILE_ALG_SIGN : nat;
18 PROFILE_ALG_SKDF : nat;
19 PROFILE_ALG_AKDF : nat;
20 }.

```

Listing 5.1: Profile Constants

5.1.2 Abstract Types

The ABSTRACT_TYPE (see Listing 5.2) contains abstract types we defined. Digest is the type of register values; DIGEST_ZERO is the digest with value zero; all the PCRs are set to DIGEST_ZERO during initialization. TSR_INIT is the profile-specified values of TSRs during initialization; the length of TSR_INIT is constrained to equal PROFILE_COUNT_TSR. We also defined different types for the key hierarchy; each type is a key or secret with different usages. PublicKey is the public part of a key; we use extract_pub to extract the public portion.

For things with different usages, we should define different types; for example, PS and DP both belong to the key hierarchy, but they have different jobs, so they are different types. This design realizes key separation: keys generated with different labels have different types; they can only be used where they should be used; the type checker refuses all the models in which keys are misused.

```

1 Record ABSTRACT_TYPE {P : PROFILE_CONSTANT} : Type := {
2   Digest : Type;
3   DIGEST_ZERO : Digest; (* digest with value zero *)
4   TSR_INIT : list Digest; (* initial value of TSRs *)
5   TSR_INIT_length : length TSR_INIT = PROFILE_COUNT_TSR P;
6
7   Context : Type; (* ctx, nonce, ... *)
8   Signature : Type; (* signature produced by CryptSign *)
9
10  (* secrets, bytes, keys (§5.5) *)
11  PrimarySeed : Type; (* PS (§5.3.2) *)
12  PS_INIT: PrimarySeed; (* initial PS (§5.3.2) *)
13  DerivationParent : Type; (* DP (§5.3.3) *)
14  DerivedBytes : Type; (* derived bytes (§8.4.1) *)

```

```

15 RestrictedKey : Type; (* attestation key (§8.5.1) *)
16 UnRestrictedKey : Type; (* signing key (§8.5.2) *)
17 PublicKey : Type; (* the public portion of the specified key (§8.4.3) *)
18 extract_pub : RestrictedKey + UnRestrictedKey -> PublicKey;
19 }

```

Listing 5.2: Abstract Types

There are some other special abstract types; we put them together into the section ABSTRACT_TYPE_EXTENSION (see Listing 5.3). Context introduces a PROFILE_CONSTANT P and an ABSTRACT_TYPE T for this section. ENV is the sampled values for TSRs; HashContext is used as the input/output type of sequence commands; Label is used in KDFs. These types will be explained in detail later.

```

1 Section ABSTRACT_TYPE_EXTENSION.
2 Context {P : PROFILE_CONSTANT}.
3 Context (T : ABSTRACT_TYPE (P := P)).
4
5 (* sampled values for TSRs (§5.3.4.3) *)
6 Record ENV : Type := {
7   TSR_SAMPLE : list (Digest T);
8   TSR_SAMPLE_length : length TSR_SAMPLE = (PROFILE_COUNT_TSR P);
9 }.
10
11 (* it turns different types into HashContext which can be fed to shc *)
12 Inductive HashContext : Type :=
13 | HC_RegSelect (n : N)
14 | HC_Digest (digest : (Digest T))
15 | HC_Context (ctx : (Context T)).
16
17 (* labels for kdf (§5.5), it specifies the input and output of a kdf *)
18 Inductive Label : Type -> Type -> Type :=
19 | MARS_LX : Label (Digest T) (DerivedBytes T)
20 | MARS_LD : Label (Digest T) (DerivationParent T)
21 | MARS_LU : Label (Context T) (UnRestrictedKey T)
22 | MARS_LR : Label (Context T) (RestrictedKey T).
23
24 End ABSTRACT_TYPE_EXTENSION.

```

Listing 5.3: Abstract Types Extension

5.1.3 Support Functions

The MARS specification assumes some support functions, and leaves the implementations to the *Profile*. We then need to abstract them by specifying the input/output types they take. Listing 5.4 shows all the support functions (except for `CryptSnapshot`); they will be invoked by commands to support the behaviors of commands.

`CryptSelfTest` takes a `(fulltest:bool)` (true means full test, and false means partial test) and returns the test result. If the self-test fails, the function returns false; otherwise, it returns true.

To do a hash process, MARS provides functions `CryptHashInit`, `CryptHashUpdate`, and `CryptHashFinal`. They are invoked by the corresponding sequence commands. But these commands usually concatenate different types together, so it would be difficult to specify the input types of these functions. To solve this problem, we introduce a new type `HashContext` (see Listing 5.3) that turns different types into `HashContext`. `HashContext` has different constructors; they take different types as input and produce a `HashContext`. Hash-related functions can then receive (list) `HashContext` as input.

KDFs are even more difficult to abstract because for different labels, they have different input and output types. To solve this problem, we define `label` as an Inductive type that takes an input type and an output type, and returns a label type. For each label, we can then specify the input and output type of KDFs (see Listing 5.3). `CryptXkdf` provides a unified interface; it uses `CryptAkdf` if `CryptAkdf` is defined, and uses `CryptSkdf` otherwise. Our model supports asymmetric KDF; `CryptAkdf` is defined. `PROFILE_LEN_XKDF` is then `PROFILE_LEN_KPRV` (the length of THE asymmetric key produced by `CryptAkdf`, see Listing 5.1).

```

1 Record SUPPORT_FUNCTION {P : PROFILE_CONSTANT} {T : ABSTRACT_TYPE (P := P)}
  : Type := {
2   CryptSelfTest : bool -> bool;
3
4   CryptHashInit : list (HashContext T);
5   CryptHashUpdate : list (HashContext T) -> (HashContext T) -> list (
      HashContext T);
6   CryptHashFinal : list (HashContext T) -> (Digest T);
7
8   CryptSign : (RestrictedKey T) + (UnRestrictedKey T) -> (Digest T) -> (
      Signature T);
9   CryptVerify : (RestrictedKey T) + (UnRestrictedKey T) -> (Digest T) -> (
      Signature T) -> bool;
10
11  CryptSkdf : forall {Input Output : Type}, (DerivationParent T) -> (Label
      T) Input Output -> Input -> Output;
12  CryptAkdf : forall {Input Output : Type}, (DerivationParent T) -> (Label

```

```

13   T) Input Output -> Input -> Output;
    CryptXkdf {Input Output : Type} (parent : (DerivationParent T)) (label :
      (Label T) Input Output) (input : Input) : Output := CryptAkdf parent
      label input;
14
15   CryptDpInit : (PrimarySeed T) -> (DerivationParent T);
16 }.

```

Listing 5.4: Support Functions

5.2 State Layer

The state layer contains the MARS state and initialization.

5.2.1 MARS_STATE

A MARS_STATE should contain state fields listed in Section 4.2; Record is a good choice here because it puts different components together into a single type, and we can access every component in it. Listing 5.5 shows the state Record. The register file is a list; the first PROFILE_COUNT_PCR registers are PCRs, and the other registers are TSRs. shc is a sequence indicator; it is an optional list; None means no sequence is running, and Some carries the list of data chunks accumulated so far.

```

1 Record MARS_STATE : Type := {
2   failure : bool;
3   PS : (PrimarySeed T);
4   DP : (DerivationParent T);
5   regs : list (Digest T);
6   shc : option (list (HashContext T))
7 }.

```

Listing 5.5: The MARS_STATE

To change the state of MARS, we use update functions. The update functions replace the corresponding field with a new value without changing other fields. Listing 5.6 shows an example of an update function.

```

1 Definition set_dp (st : MARS_STATE) (new_DP : (DerivationParent T)) :
   MARS_STATE :=
2   {
3     failure := failure st;
4     PS := PS st;
5     DP := new_DP;

```

```

6   regs := regs st;
7   shc := shc st
8   |}.

```

Listing 5.6: An Example of an Update Function

5.2.2 Initialization

MARS should be reset once the host is reset. We can consider the initialization as the start of a transition chain. Listing 5.7 shows the initialization process; PCRs are set to zero, TSRs are set to *Profile-specified* values, a full self-test is performed, and the initial DP is derived from the PS.

```

1  Definition mars_init : MARS_STATE :=
2  { | failure := negb ((CryptSelfTest F) true); (* in this model, we do full
   testing at initialization*)
3    PS := (PS_INIT T);
4    DP := (CryptDpInit F) (PS_INIT T);
5    regs := map (fun i => if i <? (PROFILE_COUNT_PCR P)
6                      then (DIGEST_ZERO T)
7                      else nth (i - PROFILE_COUNT_PCR P) (TSR_INIT T
8                                ) (DIGEST_ZERO T))
9    (List.seq 0 (PROFILE_COUNT_REG P));
   shc := None |}.

```

Listing 5.7: Initialization

5.3 Command Interface

This layer contains the concrete implementation of `CryptSnapshot` and a per-command transition function.

5.3.1 Snapshot

To model the sampling process, we use `Record` to define a type called `ENV` (environment) with a field called `TSR_SAMPLE`, and a field that restricts the length of `TSR_SAMPLE`. `TSR_SAMPLE` is the sampled values of the current environment; these values will be written into TSRs. `ENV` should be given in all the MARS commands where `CryptSnapshot` is referenced (see Listing 5.3).

The only support function for which the MARS specification describes the behavior is `CryptSnapshot`. This function takes a `regSelect` and a context as input, and computes

a snapshot of the selected registers. This is exactly where the TSRs are updated with sampled values. (Listing 5.8).

```

1  (* CryptSnapshot = CryptHash ( regSelect // REG# // ... // REG# // ctx ) *)
2  Definition CryptSnapshot (st : MARS_STATE) (env : (ENV T)) (regSelect : N)
   (ctx : (Context T)) : MARS_STATE * (Digest T) :=
3    let new_regs := sample regSelect env (regs st) in
4    let new_st := set_regs st new_regs in
5    let shc1 := (CryptHashInit F) in
6    let shc2 := (CryptHashUpdate F) shc1 ((HC_RegSelect T) regSelect) in
7    let shc3 := fold_left (CryptHashUpdate F) (map (fun i => (HC_Digest T) i)
   (sel_values regSelect (regs new_st))) shc2 in
8    let shc4 := (CryptHashUpdate F) shc3 ((HC_Context T) ctx) in
9    (new_st, (CryptHashFinal F) shc4).

```

Listing 5.8: CryptSnapshot

5.3.2 Response Code

We use Inductive to list all the response codes (see Listing 5.9). In our model, some response codes are never used. MARS_RC_IO and MARS_RC_BUFFER concern transport parsing and raw memory buffers, which do not exist at the abstraction level of a state machine. MARS_RC_COMMAND (command not supported) is also never used, since all the commands are involved in our model. MARS_CapabilityGet takes a property tag and returns the corresponding *Profile* constant. In our model, property tags are modeled as an inductive type; invalid property tags cannot be represented, so MARS_RC_VALUE is also not used.

```

1  (* response code (§6.2) *)
2  Inductive MARS_RC : Type :=
3    | MARS_RC_SUCCESS (* Command executed as expected *)
4    | MARS_RC_IO (* Input / Output or parsing error *)
5    | MARS_RC_FAILURE (* Self-testing placed MARS in failure mode or MARS is
   otherwise inaccessible *)
6    | MARS_RC_BUFFER (* Buffer pointer (null or misaligned) or length invalid
   *)
7    | MARS_RC_COMMAND (* Command not supported *)
8    | MARS_RC_VALUE (* Value out of range or incorrect for command *)
9    | MARS_RC_REG (* Invalid register index specified *)
10   | MARS_RC_SEQ. (* Sequence not started *)

```

Listing 5.9: Response Codes

5.3.3 Return Type

`ReturnType` computes the output type per command. A dependent type is a suitable choice because if there is a model in which a command would return a wrong type, the type checker then rejects this model. We can also show that sensitive types are never returned in a MARS command by checking the return type of each command, which shows the Protected Capabilities of MARS [MARS, §5.8]. Listing 5.10 shows the return type of each command.

```

1 Definition ReturnType (cmd : Command) : Type :=
2   match cmd with
3   | MARS_SelfTest _ => unit
4   | MARS_CapabilityGet _ => nat
5   | MARS_SequenceHash => unit
6   | MARS_SequenceUpdate _ => unit
7   | MARS_SequenceComplete => (Digest T)
8   | MARS_PcrExtend _ _ => unit
9   | MARS_RegRead _ => (Digest T)
10  | MARS_Derive _ _ => (DerivedBytes T)
11  | MARS_DpDerive _ _ => unit
12  | MARS_PublicRead _ _ => (PublicKey T)
13  | MARS_Quote _ _ _ => (Signature T)
14  | MARS_Sign _ _ => (Signature T)
15  | MARS_SignatureVerify _ _ _ _ => bool
16  end.

```

Listing 5.10: Return Types

5.3.4 The Transition Function

The transition function maps the current state, sampled values of the current environment, and command to the next state $s' \in S$, a response code, and an output. The commands are an inductive type with thirteen constructors. We use pattern matching to cover all the commands. Listing 5.11 shows two representative cases of the transition function. `MARS_PcrExtend` concatenates the old register value and the new digest, hashes, and stores the final digest. `MARS_Quote` takes a snapshot (which samples the selected TSRs and therefore returns an updated state), and then derives a restricted attestation key with label `MARS_LR` and signs the snapshot.

```

1 Definition transition (st : MARS_STATE) (env : ENV) (cmd : Command) :
2   MARS_STATE * MARS_RC * option (ReturnType cmd)
3   | MARS_PcrExtend pcrIndex dig =>

```

```

4   if pcrIndex <? (PROFILE_COUNT_PCR P)
5   then
6     let shc1 := (CryptHashInit F) in
7     let shc2 := (CryptHashUpdate F) shc1 ((HC_Digest T) (nth pcrIndex (
8       regs st) (DIGEST_ZERO T))) in
9     let shc3 := (CryptHashUpdate F) shc2 ((HC_Digest T) dig) in
10    let x := (CryptHashFinal F) shc3 in
11    (set_reg st pcrIndex x, MARS_RC_SUCCESS, None)
12  else (st, MARS_RC_REG, None)
13
14  | MARS_Quote regSelect nonce ctx =>
15    if valid_select regSelect
16    then
17      let (st', snapshot) := CryptSnapshot st env regSelect nonce in
18      let AK := (CryptXkdf F) (DP st') (MARS_LR T) ctx in
19      let sig := (CryptSign F) (inl AK) snapshot in
20      (st', MARS_RC_SUCCESS, Some sig)
21    else (st, MARS_RC_REG, None)

```

Listing 5.11: Two cases of the transition function: MARS_PcrExtend and MARS_Quote.

5.4 Proof Engineering

For most properties, we are only interested in the next state or the response code. We then define projections `state_of_transition` and `rc_of_transition` so that theorem statements are easier to read (Listing 5.12).

```

1  Definition state_of_transition (st : MARS_STATE) (env : (ENV T)) (cmd :
2    Command) : MARS_STATE :=
3    fst (fst (transition st env cmd)).
4
5  Definition rc_of_transition (st : MARS_STATE) (env : (ENV T)) (cmd :
6    Command) : MARS_RC :=
7    snd (fst (transition st env cmd)).

```

Listing 5.12: New state and response code of a transition

The proofs of properties all have the same flow: case-split on the failure flag, case-split on the command, then evaluate. We package the repetitive part into one tactic (see Listing 5.13). This tactic evaluates the goal and case-splits every boolean test and every option match that appears.

With this tactic, the persistence of the PS (PS is never modified) has a four-line proof (see Listing 5.14):

```

1 Ltac step :=
2   unfold state_of_transition, rc_of_transition, transition, cap_transition,
3     CryptSnapshot,
4     set_failure, set_dp, set_reg, set_regs, set_shc, set_shc_to_none;
5   simpl.
6 Ltac branches :=
7   repeat first
8     [ match goal with
9       | |- context [ if ?b then _ else _ ] => destruct b eqn:?
10      | |- context [ match ?x with Some _ => _ | None => _ end ] => destruct
11        x eqn:?
12      | |- context [ match ?x with (_, _) => _ end ] => destruct x eqn:?
13      end
14    ; simpl ].

```

Listing 5.13: Tactic

```

1 Theorem ps_persistent : forall st env cmd, PS (state_of_transition st env
2   cmd) = PS st.
3 Proof.
4   intros st env cmd.
5   destruct cmd; step; branches; try reflexivity;
6   try (destruct pt; reflexivity).
7 Qed.

```

Listing 5.14: Persistence of the PS

6 Evaluation

In Chapter 1, we split the research question into three sub-questions. In this chapter, we evaluate the formalization along the three research questions:

RQ1 Can the MARS state and its complete command interface be modeled faithfully?

RQ2 Which invariants and properties stated (or implied) by the specification can be proven?

RQ3 Which ambiguities or inconsistencies does the specification have?

6.1 Coverage of the Specification

Table 6.1 maps the sections of the MARS specification to the model. We translated the specification into our model section by section. Session management [MARS, §5.7] is the responsibility of the host; we do not model it in MARS. All thirteen commands of the command interface [MARS, §8] are modeled, regardless of whether they are mandatory, recommended, or optional [MARS, §7]. Our model also supports self-test and asymmetric cryptography.

Table 6.1: Coverage of the specification.

Specification part	Section	In the model
Cryptography	§5.1	support functions
Self-Testing	§5.2	support functions
Device State	§5.3	record MARS_STATE
Initialization	§5.4	mars_init
Key Hierarchy	§5.5	Label type, support functions
Support Functions	§5.6.1–§5.6.8	support functions
Snapshot	§5.6.9	concrete function
Protected Capabilities	§5.8	guaranteed by the structure
Profile Constants	§6.1	Record PROFILE_CONSTANT
Response Codes	§6.2	Inductive type MARS_RC
Command Interface (13 commands)	§8	transition function

As discussed in Subsection 5.3.2, some response codes are never used in our model. Besides that, during modeling, we also made our own decisions or assumptions. Especially when there were difficulties in proving a theorem, we had to refine our model. Within this boundary, the answer to RQ1 is then: our model provides broad coverage of the MARS specification, but the resulting command behavior might differ from other reasonable interpretations of the specification since we made our own decisions.

6.2 Machine-Checked Properties and Invariants

All the lemmas and nine property groups of Table 6.2 are fully proven.

Table 6.2: The nine property groups

Id	Property	Theorems
P1	the PS is never modified (§5.3.2)	1
P2	register count is preserved by every command (§5.3.4)	1
P3	failure-mode lockout (§5.3.1)	3
P4	in our model, failure mode is entered only when a self-test fails (§5.2, §8.1.1)	1
P5	sequence discipline (§8.2)	3
P6	register discipline (§5.3.4, §8.3.1)	4
P7	DP discipline (§5.3.3, §8.4.2)	1
P8	invalid regSelect rejection (§5.3.4.1)	1
P9	reachability	2

Now, we explain these properties in detail. We use the notations introduced in Section 4.1, the following properties all hold for any state $s \in S$, any environment $e \in E$, and any command $c \in C$; r stands for response code and o stands for output, just as introduced in Section 4.1.

6.2.1 PS Persistence

The PS has the highest level of protection; it can never be modified by any commands.

ps_persistent:

$$\delta_c(s, e) = (s', r, o) \implies s[\text{PS}] = s'[\text{PS}].$$

6.2.2 Register Count Invariance

The register list always has the *Profile*-specified length. This property matters because MARS_STATE itself does not constrain the length of regs. We denote n_{REG} as PROFILE_COUNT_REG.

transition_preserves_wf:

$$\delta_c(s, e) = (s', r, o) \wedge |s[\text{regs}]| = n_{\text{REG}} \implies |s'[\text{regs}]| = n_{\text{REG}}.$$

6.2.3 Failure Mode

In failure mode, the state is frozen for every command except MARS_CapabilityGet; the transition returns (st, MARS_RC_FAILURE, None); MARS_CapabilityGet still works in failure mode.

In our model, the MARS device enters the failure mode only when a self-test fails. These theorems characterize the failure mode completely.

failure_mode_freezes_state:

$$\delta_c(s, e) = (s', r, o) \wedge s[\text{failure}] = \text{true} \wedge c \neq \text{CapabilityGet} \implies s' = s.$$

failure_mode_rc:

$$\delta_c(s, e) = (s', r, o) \wedge s[\text{failure}] = \text{true} \wedge c \neq \text{CapabilityGet} \implies r = \text{RC}_{\text{failure}}.$$

capability_get_available_in_failure:

$$\delta_{\text{CapabilityGet}}(s, e) = (s', r, o) \wedge s[\text{failure}] = \text{true} \implies r = \text{RC}_{\text{success}}.$$

failure_raised_only_by_selftest:

$$\delta_c(s, e) = (s', r, o) \wedge s[\text{failure}] = \text{false} \wedge \forall c \neq \text{SelfTest} \implies s'[\text{failure}] = \text{false}.$$

6.2.4 Sequence Discipline

If MARS is not in failure mode, every non-sequence command terminates the running sequence; MARS_SequenceUpdate and MARS_SequenceComplete can only work successfully when a sequence has already been started. We denote C_{seq} as the set of sequence commands.

`nonseq_command_terminates_sequence:`

$$\delta_c(s, e) = (s', r, o) \wedge s[\text{failure}] = \text{false} \wedge c \notin \mathcal{C}_{\text{seq}} \implies s'[\text{shc}] = \perp.$$

`sequence_update_requires_start:`

$$\delta_{\text{Update}}(s, e) = (s', r, o) \wedge s[\text{failure}] = \text{false} \wedge s[\text{shc}] = \perp \implies r = \text{RC}_{\text{seq}} \wedge s' = s.$$

`sequence_complete_requires_start:`

$$\delta_{\text{Complete}}(s, e) = (s', r, o) \wedge s[\text{failure}] = \text{false} \wedge s[\text{shc}] = \perp \implies r = \text{RC}_{\text{seq}} \wedge s' = s.$$

6.2.5 Remaining Properties

It would be too tedious to write the following properties precisely using symbols, so we will just describe them in words instead. These properties also only hold when MARS is not in failure mode.

Register Discipline In the property layer, we showed that PCRs are modified only by `MARS_PcrExtend` (`pcr_modified_only_by_pcr_extend`); it rejects all the out-of-range indices (`pcr_extend_invalid_index`); `MARS_PcrExtend` does not change other PCRs (`pcr_extend_frame`); TSRs can never be extended (`pcr_extend_cannot_touch_tsr`).

DP Discipline The DP can only be replaced by the command `MARS_DpDerive`. Commands such as `MARS_Derive`, `MARS_Quote`, and `MARS_Sign` use the current DP to derive outputs or keys without replacing it: `dp_modified_only_by_dpderive`

Rejection of Invalid regSelect Some commands select registers with a bitmask `regSelect`. They always return `MARS_RC_REG` if they provide an out-of-range `regSelect`: `invalid_regselect_rejected`

Reachability `mars_init` starts a state transition chain. We proved that the PS and the length of regs are never changed: `reachable_wf`, `reachable_ps_persistent`

6.3 Findings in the Specification

The transition function in our model is total and deterministic; for every state and every command, there is an exact outcome. Therefore, it covers all the cases, including ones

that the MARS specification does not cover. This forces us to make our own decision during modeling, and any failed proof helps us refine our model and leads to a finding.

We report the three most important findings and five caveats.

- F1: invalid index in failure mode.** In failure mode, a command is invoked with an invalid register index or `regSelect`; should the response code be `MARS_RC_FAILURE` or `MARS_RC_REG`? The specification introduced both cases, but does not specify the checking order. Our model checks failure first and skips all the other checks if the command is not `MARS_CapabilityGet`.
- F2: Sequence termination in failure mode** Any non-sequence command terminates a running sequence. In failure mode, if a non-sequence command (also not `MARS_CapabilityGet`) is invoked, should this command be rejected directly or should it end the sequence first? Our model actually never reaches this case because the MARS enters failure mode if and only if a self-test fails; a self-test terminates the running sequence, so no sequence is running in failure mode. Therefore, neither interpretation affects the behavior of MARS. Our model rejects the command directly without ending the running sequence.
- F3: Termination on invalid index** Should a non-sequence command with an invalid index still terminate a running sequence? The specification says: "if other commands are used" [MARS, §8.2], but it does not say whether a command with an invalid index is considered as used when we check the index. In our thesis, we end the running sequence first and then reject the command.

These findings helped us determine the check ordering: the MARS dispatcher always checks failure mode first, and then terminates the running sequence, and finally checks if the index is valid.

- C1** In failure mode, a running sequence could be ended. `MARS_CapabilityGet` is the only command that can be accepted in failure mode, and it is a non-sequence command. If there is a running sequence in failure mode, it is possible that the sequence is terminated.
- C2** The normative C code provided in the specification does not have failure mode check or sequence termination; manufacturer must enforce these rules when they produce MARS devices.
- C3** With a NULL context, the DP is reset with `CryptDpInit` to the initial value. The input `regSelect` is not used. But in the normative C code, the command checks `regSelect` and returns `MARS_RC_REG` if it is invalid.

- C4** `MARS_SignatureVerify` [MARS, §8.5.3] verifies the signature, and returns response code `MARS_RC_SUCCESS` if the verification is successfully performed. But this response code does not mean that the result of the verification is true. It could happen that the verification fails, while it returns `MARS_RC_SUCCESS`.
- C5** The MARS emulator provided by TCG has some inconsistencies with the specification, for example, the emulator also checks the failure mode for `MARS_CapabilityGet`, and the self-test result during initialization is ignored by the emulator.

6.4 Limitations

The model shows our interpretation of the specification. Other readers could understand the specification differently, which leads to different implementations of MARS. Some theorems may not hold under other interpretations. Cryptography is abstract; we did not prove any properties of hash functions, signing functions, or KDFs. The model only covers the command behaviors, control, and transport protocols between a host and its MARS device is not modeled in our thesis.

7 Related Work

7.1 Formal Analysis of Trusted Hardware

The TPM is the most widely deployed TCG RoT; it has received the most formal attention. Gürgens et al. presented an early formal security analysis based on the TCG TPM specification. They modeled the TPM and analyzed representative scenarios such as secure boot, secure storage, and remote attestation. They identified several security issues and inconsistencies in the specifications; these findings helped refine the specifications [Gür+07]. Delaune et al. modeled several protocols that use the TPM’s PCRs, and analyzed them with the ProVerif protocol verifier [Del+11]. Properties of the TPM 2.0 HMAC authorization protocol are also formally verified in 2018 [SQF18].

7.2 Formalization of Specifications

Turning natural-language specifications into formal artifacts has developed in different areas. An executable formal model of the Arm and RISC-V Instruction Set Architectures (ISAs) is produced in the Sail language, which exposes bugs in the natural-language documents [Arm+19]. The Arm v8-M specification has also been formalized; security-relevant errors were found during analysis [Rei17].

JSCert provides a Coq formalization of the ECMAScript 5 standard; it translates the specification of JavaScript into a precise and machine-checkable model [Bod+14]. Memarian presented Cerberus; it translates an executable semantics for a large fragment of C11 into a simpler intermediate language, which makes expression and statement behavior explicit [Mem23].

7.3 Verified Implementation

Klein et al. provided the formal verification of seL4 (a general-purpose microkernel) using Isabelle/HOL. The machine-checked proof shows that the C implementation behaves correctly as the abstract specification describes, and provides strong functional-correctness guarantees. [Kle+09].

Blazy and Leroy formalized the syntax and semantics of Clight (a substantial subset of C) in Coq. This is the basis for the verification of the CompCert compiler [BL09]. CompCert is a formally verified C compiler; Coq proves that compilation from Clight (a simplified, C-like intermediate language) to assembly preserves program semantics. [Ler09].

8 Conclusion

In this thesis, we designed and implemented a formal state machine in Rocq for the natural-language MARS specification. We identified parameters of a *Profile*, including constants and support functions the specification assumes, and introduced them abstractly. To structure the *Profile* more clearly, we divided the *Profile* into `PROFILE_CONSTANT`, `ABSTRACT_TYPE`, and `SUPPORT_FUNCTION`. Our system can then be parametrized with a specific *Profile*. We also identified all state fields in MARS and represented all the MARS commands as a deterministic transition function. The transition function maps a device state and a command to a new state and a response code.

We formalized the MARS specification faithfully, but we also made our own decisions. For example, our implementation defined the check ordering of failure mode and invalid index. So the command behaviors could be different from expected. Using this model, we proved nine groups of properties and invariants of the MARS specification, such as PS is never modified, and `MARS_PcrExtend` preserves TSRs. We also reported three findings and five caveats, such as the check ordering and some errors in the MARS emulator provided by TCG. Our model can help the specification authors refine the MARS specification and help fix some bugs in the official emulator.

8.0.1 Future Work

One direction for future work is to verify a concrete implementation against the MARS specification. We can consider our model as the precise definition of the behaviors of MARS, and prove whether the concrete implementation behaves correctly as our model describes. We can use CompCert (a formally verified C compiler) [Ler09] to represent the standard C emulator in CLight. CLight is a simplified, C-like intermediate language, and it is represented by Rocq definitions in a `.v` file. And we can finally prove whether the generated CLight program refines our model.

Abbreviations

RoT Root of Trust

RTM Root of Trust for Measurement

RTR Root of Trust for Reporting

RTS Root of Trust for Storage

TCB Trusted Computing Base

TCG Trusted Computing Group

TPM Trusted Platform Module

MARS Measurement and Attestation RootS

RATS Remote ATtestation procedureS

IETF Internet Engineering Task Force

ITP Interactive Theorem Prover

AK Attestation Key

DP Derivation Parent

KDF Key Derivation Function

PCR Platform Configuration Register

PS Primary Seed

TSR Trusted Sensor Register

ISA Instruction Set Architecture

PT Property Tag

List of Figures

2.1	Passport Model [Mic25]	5
2.2	Background Check Model [Mic25]	6
2.3	MARS Key Hierarchy	8
2.4	Verifier-Host-MARS interaction model	9
3.1	System Overview	12
4.1	The Simplified MARS State Machine	22

List of Tables

2.1	Cryptographic Key Labels	8
4.1	MARS Commands (the prefix is removed for brevity)	17
6.1	Coverage of the specification.	33
6.2	The nine property groups	34

Bibliography

- [Arm+19] A. Armstrong, T. Bauereiss, B. Campbell, A. Reid, K. E. Gray, R. M. Norton, P. Mundkur, M. Wassell, J. French, C. Pulte, S. Flur, I. Stark, N. Krishnaswami, and P. Sewell. “ISA semantics for ARMv8-A, RISC-V, and CHERI-MIPS.” In: *Proceedings of the ACM on Programming Languages* 3.POPL (2019), 71:1–71:31. DOI: 10.1145/3290384.
- [Bir+23] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan. *Remote ATtestation procedureS (RATS) Architecture*. RFC 9334. Internet Engineering Task Force, Jan. 2023.
- [BL09] S. Blazy and X. Leroy. “Mechanized Semantics for the Clight Subset of the C Language.” In: *Journal of Automated Reasoning* 43 (2009), pp. 263–288. DOI: 10.1007/s10817-009-9148-3.
- [Bod+14] M. Bodin, A. Charguéraud, D. Filaretti, P. Gardner, S. Maffei, D. Naudziuniene, A. Schmitt, and G. Smith. “A Trusted Mechanised JavaScript Specification.” In: *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 2014, pp. 87–100. DOI: 10.1145/2535838.2535876.
- [Del+11] S. Delaune, S. Kremer, M. D. Ryan, and G. Steel. “Formal analysis of protocols based on TPM state registers.” In: *Proceedings of the 24th IEEE Computer Security Foundations Symposium (CSF)*. 2011, pp. 66–80. DOI: 10.1109/CSF.2011.12.
- [Gon08] G. Gonthier. “Formal Proof—The Four-Color Theorem.” In: *Notices of the American Mathematical Society* 55.11 (Dec. 2008), pp. 1382–1393.
- [Gür+07] S. Gürgens, C. Rudolph, D. Scheuermann, M. Atts, and R. Plaga. “Security evaluation of scenarios based on the TCG’s TPM specification.” In: *Proceedings of the 12th European Symposium on Research in Computer Security (ESORICS)*. 2007, pp. 438–453. DOI: 10.1007/978-3-540-74835-9_29.
- [Kim24] S. Kimmich. *What Is Remote Attestation? Enhancing Data Governance with Confidential Computing*. Confidential Computing Consortium. 2024. URL: <https://confidentialcomputing.io/2024/10/02/what-is-remote-attestation-enhancing-data-governance-with-confidential-computing/>.

- [Kle+09] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood. “seL4: formal verification of an OS kernel.” In: *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*. 2009, pp. 207–220. doi: 10.1145/1629575.1629596.
- [Ler09] X. Leroy. “Formal Verification of a Realistic Compiler.” In: *Communications of the ACM* 52.7 (2009), pp. 107–115. doi: 10.1145/1538788.1538814.
- [MARS] Trusted Computing Group. *Measurement and Attestation RootS (MARS) Library Specification*. Tech. rep. Version 1, Revision 14. Trusted Computing Group, Jan. 2023.
- [Mem23] K. Memarian. *The Cerberus C Semantics*. Tech. rep. UCAM-CL-TR-981. University of Cambridge, Computer Laboratory, May 2023. doi: 10.48456/tr-981.
- [Mic25] Microsoft. *Attestation Types and Scenarios*. 2025. URL: <https://learn.microsoft.com/en-us/azure/confidential-computing/attestation-solutions>.
- [Rei17] A. Reid. “Who Guards the Guards? Formal Validation of the Arm v8-M Architecture Specification.” In: *Proceedings of the ACM on Programming Languages* 1.OOPSLA (2017), 88:1–88:24. doi: 10.1145/3133912.
- [SQF18] J. Shao, Y. Qin, and D. Feng. “Formal analysis of HMAC authorisation in the TPM2.0 specification.” In: *IET Information Security* 12.2 (2018), pp. 133–140. doi: 10.1049/iet-ifs.2016.0005.
- [The] The Rocq Development Team. *The Rocq Prover*. Rocq. URL: <https://rocq-prover.org/>.
- [Tru] Trusted Computing Group. *What is a Root-of-Trust (RoT)?* Trusted Computing Group. URL: <https://trustedcomputinggroup.org/about/what-is-a-root-of-trust-rot/>.
- [Tru21] Trusted Computing Group. *MARS Use Cases and Considerations*. Tech. rep. Version 1, Revision 27. Trusted Computing Group, Jan. 2021.
- [Tru23] Trusted Computing Group. *Trusted Computing Group Measurement and Attestation RootS (TCG MARS) Work Group FAQ*. Tech. rep. Trusted Computing Group, Feb. 2023.
- [Tru25] Trusted Computing Group. *IoT Platform Profile of TCG Cyber Resilient Modules and Building Blocks*. Tech. rep. Version 1.0, RC 1. Trusted Computing Group, Apr. 2025.

- [Tru26] Trusted Computing Group. *TPM 2.0 Library*. Tech. rep. Version 185. Trusted Computing Group, Mar. 2026.